

ADR 3: Guardrail Implementation Approach

- **Status:** Accepted
- **Context:** LLMs are inherently probabilistic and prone to hallucination. In financial compliance, generating a rogue or uncited answer can result in regulatory fines or overlooked transaction violations. The system must guarantee that every generated output is anchored to an active regulatory clause and clearly formatted before reaching the Compliance Officer.
- **Decision:** Implement a dual-layer **Deterministic State-Machine Guardrail** using **LangGraph** structural nodes combined with an open-source evaluation wrapper (**Llama-Guard**).
- **Alternatives Considered:** * *Pure Prompt Engineering Guardrails:* Rejected because simple system prompting ("Do not hallucinate") cannot reliably prevent token manipulation or structured output syntax breakage at scale.
 - *NeMo Guardrails (Programmable Rails):* Considerably capable, but rejected due to the steep learning curve and lack of native integration with complex, asynchronous multi-agent loops.
- **Consequences:** The deterministic graph structure adds slight latency overhead because every response must clear validation checkpoints before emission. However, this structure ensures an absolute zero-tolerance threshold for uncited output text, automatically routing broken syntaxes or ambiguous validations back to a Human-in-the-Loop (HITL) manual review queue.